

Amérique du Nord – 2025 – sujet1 - Correction

Exercice 2 (6 points)

1. Méthode passer_transit

```
class Colis:
    def passer_transit(self):
        self.etat = 'transit'
```

👉 *Commentaire : la méthode modifie simplement l'attribut `etat` de l'objet.*

2. Fonction ajouter_colis avec contrainte de poids

```
def ajouter_colis(liste, colis):
    if colis.poids <= 25:
        liste.append(colis)
    else:
        print("Dépassement du poids maximal autorisé")
```

👉 *Commentaire : on teste le poids avant l'ajout à la liste.*

3. Fonction nb_colis

```
def nb_colis(liste):
    return len(liste)
```

👉 *Commentaire : la longueur de la liste correspond au nombre de colis.*

4. Fonction poids_total

```
def poids_total(liste):
    total = 0
    for colis in liste:
        total = total + colis.poids
    return total
```

👉 *Commentaire : on additionne les poids de tous les colis de la liste.*

5. Fonction liste_colis_etat

```
def liste_colis_etat(liste, statut):  
    resultat = []  
    for colis in liste:  
        if colis.etat == statut:  
            resultat.append(colis)  
    return resultat
```

👉 *Commentaire : on construit une nouvelle liste contenant uniquement les colis ayant l'état demandé.*

6. Tri décroissant et complexité

Le tri utilisé est un tri par sélection.
Son coût dans le pire des cas est en $O(n^2)$.

7. Autre algorithme

Un autre algorithme possible est le tri fusion, son coût dans le pire des cas est en **$O(n \log n)$** .

8. Fonction récursive chargement_glouton

```
def chargement_glouton(liste, rang, charge):  
    if rang == len(liste):  
        return []  
    if liste[rang].poids <= charge:  
        return [liste[rang]] + chargement_glouton(  
            liste, rang + 1, charge - liste[rang].poids)  
    else:  
        return chargement_glouton(liste, rang + 1, charge)
```

👉 *Commentaire : on teste récursivement chaque colis en partant du plus lourd.*

9. Origine possible de l'erreur RecursionError

Cette erreur peut apparaître si le cas de base n'est jamais atteint, par exemple si `rang` n'est pas incrémenté lors des appels récursifs. La profondeur de récursion dépasse alors la limite autorisée par Python.

10. Version itérative chargement_glouton2

```
def chargement_glouton2(liste, capacite):  
    colis_a_charger = []  
    charge = capacite  
    for colis in liste:  
        if colis.poids <= charge:  
            colis_a_charger.append(colis)  
            charge = charge - colis.poids  
    return colis_a_charger
```

👉 *Commentaire : cette version évite la récursivité et applique directement l'algorithme glouton à l'aide d'une boucle for.*
